

Q1. Objective Questions

I) What does the instruction “MOV A, B” mean in assembly language?

Correct Answer:

(c) Move the contents of register B to register A

Explanation:

MOV A, B copies data from register B into register A.

After execution:

$$A \leftarrow B$$

II) Which standard is used to represent floating-point numbers?

Correct Answer:

(b) IEEE 754

Explanation:

IEEE 754 defines the standard format for floating-point arithmetic in computers.

It specifies:

- Sign bit
 - Exponent
 - Mantissa
-

III) In a pipelined CPU, which stage executes the instruction?

Correct Answer:

(c) Execute

Explanation:

In the Execute stage, the CPU performs:

- Arithmetic operations
 - Logical operations
 - Address calculations
-

IV) Which is a type of bus used in computer systems?

Correct Answer:

(d) All of the above

Explanation:

Computer systems use:

- Power bus
- Control bus
- Data bus

All are important for communication between hardware components.

V) Which register holds the result of arithmetic or logical operation?

Correct Answer:

(b) Accumulator (AC)

Explanation:

Accumulator stores:

- Intermediate results
- Final arithmetic/logic operation results

Q.2. What is virtual memory and how does it work? How does it enhance memory management and allow for efficient multitasking?CO3 L3 [5]**Virtual Memory**

Virtual memory is a memory management technique in which the operating system uses a part of secondary storage (hard disk/SSD) as an extension of main memory (RAM).

It allows a computer to execute programs larger than the available physical memory.

Working of Virtual Memory**1. Division into Pages and Frames**

- The program is divided into fixed-size blocks called pages.
- RAM is divided into fixed-size blocks called frames.

2. Loading Pages into RAM

Only the required pages of a program are loaded into RAM.

The remaining pages stay in secondary storage.

3. Address Translation

When the CPU generates a virtual address:

- The operating system checks the page table.
- The virtual address is converted into a physical address.

4. Page Fault Handling

If the required page is not present in RAM:

- A page fault occurs.
- The operating system loads the required page from disk into RAM.
- If RAM is full, an old page is removed using a page replacement algorithm.

Diagram of Virtual Memory

CPU



Virtual Address



Page Table



Physical Memory (RAM)



Secondary Storage (Disk)

How Virtual Memory Enhances Memory Management**1. Efficient Use of RAM**

Only active pages are loaded into memory.

Unused pages remain on disk.

This reduces memory wastage.

2. Allows Execution of Large Programs

Programs larger than physical RAM can run smoothly using virtual memory.

3. Better Memory Allocation

Each process gets its own virtual address space.

This improves organization and protection.

How Virtual Memory Supports Efficient Multitasking

1. Multiple Programs Run Simultaneously

The operating system can keep parts of many programs in memory at the same time.

This improves multitasking performance.

2. Process Isolation

Each process works in its own virtual memory space.

One program cannot directly affect another.

This improves security and stability.

3. Faster Context Switching

The OS can switch between processes efficiently without loading entire programs again.

Advantages of Virtual Memory

- Supports multitasking
 - Runs large applications
 - Better memory utilization
 - Improves system flexibility
 - Provides process protection
-

Disadvantages

- Accessing disk is slower than RAM
 - Excessive paging reduces performance (Thrashing)
-

Conclusion

Virtual memory is an important memory management technique that increases system efficiency by extending RAM using secondary storage. It enables efficient multitasking, better memory utilization, and execution of large programs in modern computer systems.

Q.3. Describe the different algorithms for multiplying signed and unsigned numbers.

Explain how hardware implementations optimize multiplication operations. CO4 L4

[CO4 L4] [5 Marks]

Introduction

Multiplication is an important arithmetic operation in computer systems. Different algorithms are used for multiplying signed and unsigned binary numbers efficiently. Hardware techniques are also used to improve multiplication speed and processor performance.

1. Unsigned Binary Multiplication

Unsigned multiplication is used for positive binary numbers only.

The common method used is the Shift-and-Add Algorithm.

Shift-and-Add Algorithm

Steps

1. Check the least significant bit (LSB) of the multiplier.
 2. If the bit is 1, add the multiplicand to the partial product.
 3. Shift multiplicand left by one bit.
 4. Shift multiplier right by one bit.
 5. Repeat until all bits are processed.
-

Example

Multiply:

```
[
101_2 \times 11_2
]
[
101_2 = 5, \quad 11_2 = 3
]
  101
\times 11
-----
  101
+ 1010
-----
 1111
[
1111_2 = 15
]
```

Advantages

- Simple implementation
 - Suitable for unsigned numbers
-

Disadvantages

- Slow for large numbers
 - Requires many addition operations
-

2. Signed Binary Multiplication

Signed multiplication handles both positive and negative numbers.

The commonly used method is Booth's Algorithm.

Booth's Algorithm

Booth's algorithm multiplies signed binary numbers represented in 2's complement form.

It reduces unnecessary addition and subtraction operations.

Working Principle

The algorithm checks:

- Current bit (Q₀)

- Previous bit (Q_{-1})

Operations:

($Q_0 Q_{-1}$) Operation

00	No operation
01	Add multiplicand
10	Subtract multiplicand
11	No operation

After each step, arithmetic right shift is performed.

Advantages of Booth's Algorithm

- Handles signed numbers efficiently
- Faster multiplication
- Reduces arithmetic operations

Hardware Optimization Techniques

Modern processors use hardware optimization methods to improve multiplication performance.

1. Parallel Multipliers

Multiple bits are processed simultaneously.

Benefit:

- Faster execution

2. Pipelining

Multiplication process is divided into stages.

Benefit:

- Increases throughput

3. Wallace Tree Multiplier

Uses parallel adders to reduce partial products quickly.

Benefit:

- High-speed multiplication

4. Booth Encoding

Optimized version of Booth's algorithm.

Benefit:

- Fewer partial products
- Reduced hardware complexity

Conclusion

- Shift-and-Add algorithm is used for unsigned multiplication.
- Booth's Algorithm is used for signed multiplication.
- Hardware techniques like pipelining, parallel multipliers, and Wallace trees improve multiplication speed and processor performance.

Q.4 Discuss the challenges in implementing pipelining, such as data hazards and control hazards. How are these hazards resolved in modern processors?

[CO5 L4] [5 Marks]

Introduction

Pipelining is a technique in which multiple instructions are executed simultaneously in different stages of a processor. It improves CPU throughput and performance.

However, pipelining introduces several challenges called pipeline hazards.

Types of Pipeline Hazards

1. Data Hazards

Data hazards occur when one instruction depends on the result of another instruction that has not completed yet.

Example

ADD R1, R2, R3

SUB R4, R1, R5

The second instruction needs the value of R1 before the first instruction finishes execution.

Types of Data Hazards

a) RAW (Read After Write)

Occurs when an instruction tries to read data before it is written.

b) WAR (Write After Read)

Occurs when writing happens before reading.

c) WAW (Write After Write)

Occurs when two instructions write to the same location in the wrong order.

Solutions for Data Hazards

1. Operand Forwarding

The result is directly forwarded from one pipeline stage to another without waiting for write-back.

Benefit:

- Reduces delay
-

2. Pipeline Stall

The processor temporarily pauses execution until data becomes available.

Benefit:

- Ensures correct execution
-

3. Register Renaming

Different physical registers are assigned to avoid conflicts.

Benefit:

- Removes false dependencies
-

2. Control Hazards

Control hazards occur because of branch and jump instructions.

The processor may not know which instruction to fetch next.

Example

IF ($A > B$)

GOTO LABEL

The next instruction depends on branch outcome.

Solutions for Control Hazards

1. Branch Prediction

The processor predicts whether a branch will be taken or not.

Benefit:

- Reduces waiting time
-

2. Delayed Branching

Useful instructions are placed immediately after branch instructions.

Benefit:

- Minimizes pipeline stalls
-

3. Speculative Execution

Processor executes predicted instructions before branch result is confirmed.

Benefit:

- Improves performance
-

3. Structural Hazards

Structural hazards occur when hardware resources are insufficient.

Example:

- Two instructions need the same memory simultaneously.
-

Solutions for Structural Hazards

- Separate instruction and data memory
 - Multiple functional units
 - Resource duplication
-

Hazard Handling in Modern Processors

Modern processors use advanced techniques such as:

- Dynamic scheduling
- Out-of-order execution
- Branch prediction
- Speculative execution
- Superscalar architecture

These methods improve processor efficiency and reduce pipeline delays.

Conclusion

Pipelining improves CPU performance but introduces hazards such as:

- Data hazards
- Control hazards
- Structural hazards

Modern processors resolve these hazards using forwarding, stalling, branch prediction, and advanced execution techniques to achieve high throughput and efficient processing.

Q.5 What is pipelining and how does it improve instruction throughput? Explain the different stages of a typical instruction pipeline.

[CO5 L6]

Introduction

Pipelining is an important technique used in modern processors to increase CPU performance and instruction throughput.

It allows multiple instructions to execute simultaneously by dividing instruction execution into different stages.

It works similar to an assembly line in a factory.

What is Pipelining?

Pipelining is a technique in which:

- The execution of an instruction is divided into several stages.
- Different instructions are processed in different stages at the same time.

Thus, the CPU can work on multiple instructions simultaneously.

How Pipelining Improves Instruction Throughput

Without Pipelining

- One instruction completes before the next starts.
 - CPU resources remain idle during some operations.
 - Execution becomes slower.
-

With Pipelining

- Multiple instructions overlap in execution.
- Different stages work in parallel.
- More instructions are completed in less time.

Hence:

[
\boxed{\text{Instruction throughput increases}}
]

Example of Pipelining

Instruction 1 : IF → ID → EX → MEM → WB

Instruction 2 : IF → ID → EX → MEM → WB

Instruction 3 : IF → ID → EX → MEM → WB

Here:

- While one instruction executes,
- another instruction is decoded,
- and another is fetched.

This parallelism increases CPU efficiency.

Stages of a Typical Instruction Pipeline

A typical pipeline consists of five stages.

1. Instruction Fetch (IF)

- The CPU fetches the instruction from memory.
- Program Counter (PC) stores the address of the next instruction.

Operation:

[
\boxed{\text{Fetch instruction from memory}}
]

2. Instruction Decode (ID)

- The fetched instruction is decoded.
- CPU identifies:
 - operation type
 - source registers
 - destination registers

Operation:

[
\boxed{\text{Decode instruction and read operands}}
]

3. Execute (EX)

- Arithmetic and logical operations are performed.
- ALU executes the instruction.

Examples:

- Addition
- Subtraction
- Comparison

Operation:

[
\boxed{\text{Perform computation}}
]

4. Memory Access (MEM)

- Access memory if needed.
- Read or write data from/to memory.

Used in:

- Load instructions
- Store instructions

Operation:

[
\boxed{\text{Access data memory}}
]

5. Write Back (WB)

- Final result is written into destination register.

Operation:

[
\boxed{\text{Store result into register}}
]

Pipeline Diagram

Stage 1 → Instruction Fetch (IF)

Stage 2 → Instruction Decode (ID)

Stage 3 → Execute (EX)

Stage 4 → Memory Access (MEM)

Stage 5 → Write Back (WB)

Advantages of Pipelining

1. Increased Throughput

More instructions completed per unit time.

2. Better CPU Utilization

All processor units remain active.

3. Faster Program Execution

Instructions execute concurrently.

4. Improved Performance

Overall processor efficiency increases.

Disadvantages of Pipelining

- Pipeline hazards may occur.
 - Complex hardware design.
 - Branch instructions may cause delays.
-

Conclusion

Pipelining is a technique that improves processor performance by executing multiple instructions simultaneously in different stages. The main stages are:

1. Instruction Fetch
2. Instruction Decode
3. Execute
4. Memory Access
5. Write Back

By overlapping instruction execution, pipelining increases instruction throughput and enhances overall CPU efficiency.